

第十章：进化调度

引言

本章导读

本章是 AI 小镇项目的**基础设施层**，是一次在教学操作系统领域的**原创性探索**。我们**首次在 uCore 教学操作系统中实现了"进化调度"机制**，将自然界的生存竞争法则引入到进程调度中，为后续 NPC 社交系统奠定了基础。

本章的创新贡献：

- 定义了"NPC生命周期模型"**：创建→演化→终止→重生，将每个 NPC 作为独立进程运行，为 AI 驱动的虚拟世界提供了进程抽象。
- 设计了"活力演化"双向机制**：
 - 活力增加：正向交流、繁衍（ch11 实现 IPC 奖励）
 - 活力减少：随时间每 tick-1，负向交流
- 实现了"OOM Killer"淘汰机制**：优先级归零的 NPC 被系统自动淘汰，模拟自然选择压力。
- 建立了可观测性接口**：提供状态查询系统调用，便于监控 NPC 生存状况。

在前面的章节中，我们实现的调度器都是"一视同仁"的——所有进程平等地竞争 CPU 时间。但在现实世界（以及我们想要构建的 AI 小镇）中，存在着生存竞争：资源有限，适者生存。本章将这种自然法则引入操作系统调度。

从调度器到"自然选择"

传统的进程调度关注的是公平性和效率：

- 时间片轮转保证每个进程都有机会运行
- 优先级调度让重要任务先执行

但我们想要的是一个**动态演化的系统**：

- NPC 的优先级随时间自然衰减（模拟资源消耗）
- 有效的社交行为可以获得奖励（模拟合作互利）
- 无法维持活力的 NPC 被淘汰（模拟自然选择）

这就是"进化调度"的核心理念。

本章目标

本章将实现一个进化调度系统，包括：

组件	功能
NPC 注册	将普通进程标记为 NPC，赋予初始活力
优先级衰减	每个时钟 tick，NPC 活力自动减 1
OOM Killer	活力归零的 NPC 被系统终止
状态查询	NPC 可查询自身存活状态和全局信息

组件	功能
IPC 奖励接口	为 ch11 的社交奖励预留框架

实践体验

获取本章代码：

源码已上传大赛官网

在 qemu 模拟器上运行本章代码：

```
$ make BASE=1 CHAPTER=10
$ make run
>> ch10_world
[world] Initializing NPC world...
[NPC 0] Born!
[NPC 1] Born!
[NPC 2] Born!
...
[OOM killer] NPC 0 killed (prio=0)
[NPC 0] I'm dying...
[world] All NPCs died, world ended
```

看到 NPC 依次诞生、存活、最终因活力耗尽而死亡，说明进化调度系统工作正常！

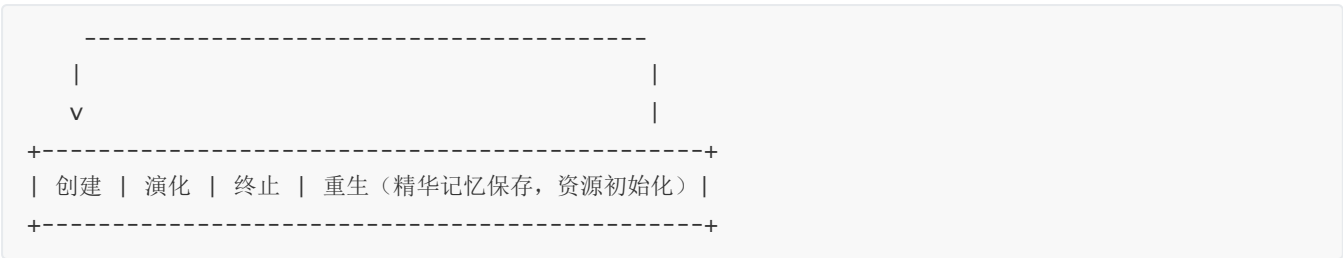
本章代码树

```
os-btbu/
├─ os/
│   ├── ai_sched.c      # 进化调度核心实现
│   ├── ai_sched.h      # 调度参数和结构体定义
│   └─ syscall.c        # 系统调用处理（新增3个syscall）
└─ user/
    ├── src/
    │   ├── ch10_world.c  # 世界管理器
    │   └─ ch10_npc.c    # NPC进程
    ├── lib/
    │   └─ ai_sched.c     # 用户态syscall封装
    └─ include/
        └─ ai_sched.h     # 用户态头文件
```

NPC生命周期模型

设计理念

我们将 NPC 的生命抽象为四个阶段：



- **创建**：通过 `fork` + `exec` 生成新 NPC 进程，调用 `npc_register` 注册
- **演化**：NPC 在世界中活动，优先级随时间衰减，通过社交获得奖励
- **终止**：优先级归零时被 OOM Killer 终止
- **重生**：（未来扩展）保留核心记忆，重新初始化资源

进程控制块扩展

为支持 NPC 机制，需要在 PCB 中添加新字段：

```
/* os/proc.h */
struct proc {
    // ... 原有字段 ...

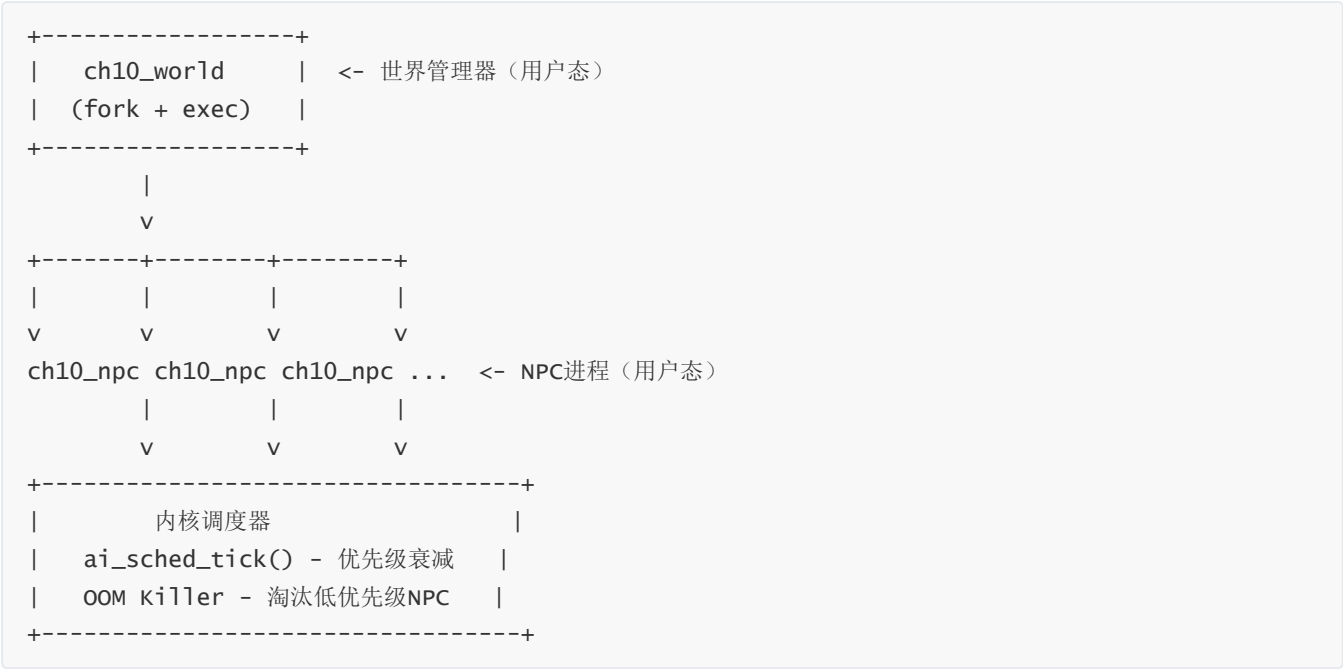
    /* ch10: NPC进化调度相关 */
    int is_npc;           /* 是否为NPC进程 */
    int npc_id;           /* NPC的唯一标识符 */
    int dynamic_prio;     /* 动态优先级（活力值） */
};
```

调度参数

```
/* os/ai_sched.h */
#define NPC_INIT_PRIO      50    /* 初始优先级 */
#define NPC_DECAY_RATE    1     /* 每tick衰减量 */
#define NPC_IPC_REWARD_BYTES 64 /* IPC奖励阈值（字节） */
#define NPC_DEATH_THRESHOLD 0   /* 死亡阈值 */
#define NPC_PRIO_MAX      100   /* 最大优先级 */
```

系统架构

整体结构



世界管理器

世界管理器负责创建和监控所有 NPC：

```
/* user/src/ch10_world.c */
int main() {
    printf("[world] Initializing NPC world...\n");

    /* 创建3个NPC进程 */
    for (int i = 0; i < 3; i++) {
        int pid = fork();
        if (pid == 0) {
            char id_str[8];
            int_to_str(i, id_str);
            char *argv[] = {"ch10_npc", id_str, 0};
            exec("ch10_npc", argv);
            exit(-1);
        }
    }

    /* 等待所有NPC死亡 */
    while (wait(0) > 0);

    printf("[world] All NPCs died, world ended\n");
    return 0;
}
```

NPC 进程

每个 NPC 进程的基本行为：

```
/* user/src/ch10_npc.c */
int main(int argc, char *argv[]) {
    int npc_id = str_to_int(argv[1]);
    struct npc_status st;

    printf("[NPC %d] Born!\n", npc_id);

    /* 向内核注册为NPC */
    npc_register(npc_id);

    /* 主循环 */
    while (1) {
        npc_get_status(&st);

        /* 检查是否死亡 */
        if (st.my_prio <= 0) {
            printf("[NPC %d] I'm dying...\n", npc_id);
            break;
        }

        /* ch11: 这里将添加社交行为 */

        npc_yield(); /* 触发进化调度 */
    }

    return 0;
}
```

系统调用接口

本章新增三个系统调用：

系统调用	调用号	功能
<code>npc_register</code>	483	注册当前进程为NPC
<code>npc_get_status</code>	484	获取NPC状态信息
<code>npc_yield</code>	485	主动让出CPU，触发进化调度

npc_register

```
int npc_register(int npc_id);
```

功能：将当前进程注册为 NPC，赋予初始优先级。

参数：

- `npc_id`: NPC 的唯一标识符

返回值:

- 0: 成功
- -1: 失败 (如已注册过)

npc_get_status

```
int npc_get_status(struct npc_status *st);
```

功能: 获取当前 NPC 的状态信息。

npc_status 结构体:

```
struct npc_status {  
    int tick;           /* 当前系统时钟 */  
    int alive_npcs;     /* 存活NPC数量 */  
    int my_prio;        /* 本NPC当前优先级 */  
};
```

返回值:

- 0: 成功
- -1: 失败 (非 NPC 进程调用)

npc_yield

```
void npc_yield(void);
```

功能: NPC 主动让出 CPU, 触发 `ai_sched_tick()`, 执行优先级衰减和 OOM 检查。

核心算法

优先级衰减

这是进化调度的核心。每次调用 `ai_sched_tick()` 时, 所有 NPC 的优先级都会衰减:

```
/* os/ai_sched.c */  
void ai_sched_tick(void) {  
    struct proc *p;  
  
    for (p = proc; p < &proc[NPROC]; p++) {  
        acquire(&p->lock);  
  
        /* ch10: 跳过非NPC进程 */  
        if (!p->is_npc || p->state == UNUSED) {  
            release(&p->lock);  
            continue;  
        }  
    }
```

```

/* ch10: 跳过已死亡的NPC */
if (p->dynamic_prio < 0) {
    release(&p->lock);
    continue;
}

/* ch10: 优先级衰减 */
p->dynamic_prio -= NPC_DECAY_RATE;

/* ch10: OOM killer */
if (p->dynamic_prio <= NPC_DEATH_THRESHOLD) {
    printf("[OOM killer] NPC %d killed (prio=%d)\n",
        p->npc_id, p->dynamic_prio);
    p->dynamic_prio = -1; /* 标记为已死亡 */
    p->killed = 1;
}

release(&p->lock);
}

global_tick++;
}

```

关键点:

1. 遍历所有进程，只处理 NPC 进程
2. 每次调用使优先级减少 `NPC_DECAY_RATE` (默认为 1)
3. 当优先级降至阈值以下时，触发 OOM Killer
4. 被杀死的 NPC 标记 `killed = 1`，等待进程清理

IPC 奖励 (ch11 预留)

```

/* os/ai_sched.c */
void ai_sched_ipc_reward(struct proc *sender, struct proc *receiver, int bytes) {
    if (bytes >= NPC_IPC_REWARD_BYTES) {
        int reward = bytes / NPC_IPC_REWARD_BYTES;

        sender->dynamic_prio += reward;
        if (sender->dynamic_prio > NPC_PRIO_MAX)
            sender->dynamic_prio = NPC_PRIO_MAX;

        receiver->dynamic_prio += reward;
        if (receiver->dynamic_prio > NPC_PRIO_MAX)
            receiver->dynamic_prio = NPC_PRIO_MAX;
    }
}

```

这个函数在 ch11 中会被调用，当 NPC 之间进行有效通信时，双方都获得优先级奖励。

测试结果

运行 `ch10_world` 的典型输出：

```
[World] Initializing NPC world...
[NPC 0] Born!
[NPC 1] Born!
[NPC 2] Born!
[NPC 0] tick=5, prio=45, alive=3
[NPC 1] tick=5, prio=45, alive=3
[NPC 2] tick=5, prio=45, alive=3
...
[NPC 0] tick=45, prio=5, alive=3
[NPC 1] tick=45, prio=5, alive=3
[OOM Killer] NPC 0 killed (prio=0)
[NPC 0] I'm dying... (prio=0)
[OOM Killer] NPC 1 killed (prio=0)
[NPC 1] I'm dying... (prio=0)
[OOM Killer] NPC 2 killed (prio=0)
[NPC 2] I'm dying... (prio=0)
[World] AllNPCs died, world ended
```

所有 NPC 经过约 50 轮后因优先级耗尽而死亡，符合预期设计。

知识点总结

1. **进化调度**：将自然选择法则引入进程调度，实现动态的生存竞争
2. **优先级衰减**：
 - 模拟资源消耗和时间流逝
 - 迫使 NPC 采取行动（社交）来维持生存
3. **OOM Killer**：
 - 淘汰无法维持活力的进程
 - 释放系统资源给更"适应"的个体
4. **IPC 奖励**：
 - 鼓励进程间协作
 - 社交行为带来生存优势
5. **可观测性**：
 - 提供状态查询接口
 - 便于调试和监控

思考题

1. 如何在此章节实现"繁衍"机制（NPC 产生子进程）？
2. 如何实现 NPC 的"重生"机制？当优先级归零后，保留核心记忆重新初始化资源？
3. 当前的 OOM Killer 是"一刀切"的，如何实现更公平的淘汰机制（如随机淘汰低优先级 NPC）？

下一步：ch11 NPC社交系统

在实现了进化调度基础设施之后，下一章我们将实现 NPC 之间的社交通信机制。NPC 通过有效社交可获得优先级奖励，延长生存时间——这正是"合作带来生存优势"的自然法则在操作系统中的体现。

```
/* ch11 新增系统调用预告 */  
int npc_send(int to_npc_id, char *msg, int len);  
int npc_recv(char *buf, int maxlen, int *from_npc_id);
```